Release 2026-11-09: Windows laptop and VM

What to do on the Windows laptop (Command Prompt) and the Ubuntu VM (bash) to bring them to today's code.

On the VM: **Postgres stays the database** (`DB_BACKEND=postgres`). The Cosmos key there is read-only, and every mode that uses Cosmos writes to it. **Chart runs go through Service Bus** (`RUN_MODE=queue`): the API queues each chart and the `worker` containers run them.

Order: download → push → VM pull → back up → settings → database → restart → check → re-run charts → utilities.
Database: container `postgres-db` on the VM, user `aiuser`, databases `imaging_outputs` (ours) and `imaging_outputs_client`.

---

1. Windows laptop: download from GitHub and copy into Azure DevOps

The full commands, and the `.gitignore` to keep as `C:\work\devops.gitignore` (once), are in VM_GUIDE §5.1. In short:

```
:: Windows (Command Prompt)
:: 1. github.com/abhinavdg-ever/model-repo -> Code -> Download ZIP, then unpack it
cd /d %USERPROFILE%\Downloads
rmdir /s /q model-repo-main 2>nul
tar -xf model-repo-main.zip

:: 2. Copy into the DevOps folder. /MIR makes it match GitHub (deleted files go too); the
::    /XD folders (git, venvs, weights, logs, chart output, review data, db-utilities input)
::    and .env files are neither copied nor deleted. Full paths: other folders share names.
set SRC=%USERPROFILE%\Downloads\model-repo-main
set DST=C:\work\advantmed-devops
robocopy "%SRC%" "%DST%" /MIR /NFL /NDL /NP /XF .env ._* .DS_Store /XD .git .venv .venv-test
node_modules "%SRC%\core-pipeline\models" "%DST%\core-pipeline\models" "%SRC%\core-pipeline\logs" "%DST%\core-
pipeline\logs" "%SRC%\core-pipeline\outputs" "%DST%\core-pipeline\outputs" "%SRC%\review-ui\data" "%DST%\review-
ui\data" "%SRC%\db-utilities\input" "%DST%\db-utilities\input"

:: 3. The small model files GitHub keeps (heads, configs, READMEs): added, never deleting
::    the big weights already in the DevOps folder.
robocopy "%SRC%\core-pipeline\models" "%DST%\core-pipeline\models" /E /NFL /NDL /NP /XF ._*

:: 4. Your DevOps .gitignore (the GitHub one was just copied over it)
copy /y C:\work\devops.gitignore "%DST%\.gitignore"

:: 5. Push C:\work\advantmed-devops to Azure DevOps as usual
```

robocopy exit codes 0–7 are success (1 = files copied); 8 or more is a failure.

Once: save this as `C:\work\devops.gitignore` (step 4 copies it in each release). It keeps training, scripts, tests, `.env` files, big weights and run output out of Azure DevOps:

```
# Azure DevOps repo .gitignore (kept as C:\work\devops.gitignore, copied in each release).
# GitHub keeps everything; DevOps and the VM get only what runs.

# --- Development only: on GitHub, never on the VM ---
training/
scripts/
tests/
.venv-test/
CLAUDE.md
PLAN.md

# --- Secrets: each host keeps its own .env (made from .env.example) ---
.env
```

```
**/.env
!**/.env.example

# --- Weights: the big ones are copied onto the VM by hand, never committed. The small
#     files GitHub keeps (joblib heads, heads.pt, the hw pkl) DO go through.
*.safetensors
*.onnx
*.bin
*.pth
core-pipeline/models/blank-junk/bert/
core-pipeline/models/ner/
core-pipeline/models/rapidocr/
core-pipeline/models/docling/
core-pipeline/models/semantic-model/

# --- Written at run time ---
core-pipeline/logs/**
!core-pipeline/logs/README.md
core-pipeline/outputs/charts/*
!core-pipeline/outputs/charts/.gitkeep
core-pipeline/outputs/metadata/*
!core-pipeline/outputs/metadata/.gitkeep
utilities/output/
db-utilities/input/*
!db-utilities/input/.gitkeep
*.log

# --- Tools and OS files ---
.venv/
**/.venv/
node_modules/
**/node_modules/
__pycache__/
*.pyc
.DS_Store
._*
```

2. Windows laptop: the client UI

github.com/abhinavdg-ever/ui-temp → **Code** → **Download ZIP**, and deliver it the way you usually do (on the VM its folder updates with `git pull && docker compose up -d --build`).

No new model weights this release: nothing to copy into `core-pipeline/models/`.

3. VM: pull

```
# Ubuntu VM
cd ~/advantmed-imaging-pipeline
git pull                                    # from Azure DevOps
```

4. VM: back up the database (Postgres in Docker)

The database runs in the Docker container **`postgres-db`** on the VM (172.20.4.170:5432): `imaging_outputs` is ours, `imaging_outputs_client` the client's. The ground-truth copy in §9 replaces rows and the re-run in §8 rewrites verdicts, so back both up first.

```
# Ubuntu VM
docker exec postgres-db pg_dump -U postgres -d imaging_outputs -Fc > ~/backup_$(date +%Y%m%d_%H%M).dump
docker exec postgres-db pg_dump -U postgres -d imaging_outputs_client -Fc > ~/backup_client_$(date
```

```
+%Y%m%d_%H%M).dump
        ls -lh ~/backup_*.dump | tail -2                                  # neither should be tiny
```

To look inside: `docker exec -it postgres-db psql -U postgres -d imaging_outputs` (`\q` to leave).

To put a backup back (only if something went wrong):

```
        docker exec -i postgres-db pg_restore -U postgres -d imaging_outputs --clean --if-exists <
~/backup_<stamp>.dump
```

5. VM: settings

The `.env` files are not in git. Add any line below that a file does not have yet, in the section named; keep each `.env` in the same order as its `.env.example`.

`core-pipeline/.env`

```
        # --- 14. PLATFORM ---
        DB_BACKEND=postgres
        COSMOS_ENDPOINT=
        COSMOS_KEY=
        COSMOS_DATABASE=imaging
        SERVICEBUS_CONNECTION_STRING=<the Service Bus connection string>
        SERVICEBUS_QUEUE=chart-runs
        ORCHESTRATOR_API_KEY=
        API_KEY_REQUIRED=false
        RUN_MODE=queue
        WORKER_LOCK_RENEW_HOURS=6

        # --- 15. RESOURCES ---
        RESOURCES_FROM_BLOB=false
        RESOURCES_CONTAINER=imaging-models
        RESOURCES_MODELS=
        RESOURCES_CANON_FROM_BLOB=false
```

Remove `METADATA_ROOT=` if it is still there (no longer read). The queue `chart-runs` must have **sessions enabled** (it was created that way).

`review-ui/.env` (and the client UI's `.env`, the same lines)

```
        # --- 6. PLAYGROUND ---  (after PLAYGROUND_CONTAINER)
        ORCHESTRATOR_API_KEY=
        # --- 8. PLATFORM ---
        DB_BACKEND=postgres
        COSMOS_ENDPOINT=
        COSMOS_KEY=
        COSMOS_DATABASE=imaging
```

`db-utilities/.env` (new folder: make it once)

```
        cd ~/advantmed-imaging-pipeline/db-utilities
        cp .env.example .env
        # DATABASE_URL and AZURE_STORAGE_* as in core-pipeline/.env; DB_BACKEND=postgres
```

Check that the keys match the examples (prints key names only, never values):

```
        cd ~/advantmed-imaging-pipeline
        for d in core-pipeline review-ui db-utilities; do
          diff <(grep -o '^[A-Z_]*=' $d/.env.example) <(grep -o '^[A-Z_]*=' $d/.env) >/dev/null \
            && echo "$d: same keys" || { echo "$d: differs"; diff <(grep -o '^[A-Z_]*=' $d/.env.example) <(grep
-o '^[A-Z_]*=' $d/.env); }
        done
```

6. VM: database upgrade

`migrate.sql` pulls in the files under `schema/migrations/`, so the folder is copied into the container first. Safe to run every release. Both databases: the client's review UI reads the same tables.

```
        # Ubuntu VM
        cd ~/advantmed-imaging-pipeline
        docker cp schema postgres-db:/tmp/schema
        docker exec postgres-db psql -U postgres -d imaging_outputs        -v ON_ERROR_STOP=1 -f
/tmp/schema/migrate.sql
        docker exec postgres-db psql -U postgres -d imaging_outputs_client -v ON_ERROR_STOP=1 -f
/tmp/schema/migrate.sql
        docker exec postgres-db psql -U postgres -d imaging_outputs -c "SELECT count(*) AS stages FROM
pipeline_stage;"   # 15
```

7. VM: restart and check

```
        # Ubuntu VM
        # core-pipeline: new Python packages (azure-cosmos, azure-servicebus), so --build.
        # Each worker runs one chart at a time with its own models (several GB): start with 2.
        cd ~/advantmed-imaging-pipeline/core-pipeline
        docker compose up -d --build --scale worker=2
        curl -fsS localhost:8001/ready && echo pipeline ready
        curl -s localhost:8001/health | python3 -c "import json,sys; h=json.load(sys.stdin); print('db',
h.get('db_backend')); print('run_mode', h.get('run_mode')); print('queue', h.get('queue'))"
        # expect: db {'configured': 'postgres', 'active': 'postgres', ...}, run_mode queue, the queue reachable
        docker compose logs --tail 20 worker                                  # "queue worker starting"

        # review-ui
        cd ../review-ui
        docker compose up -d --build
        curl -fsS localhost:4000/docs >/dev/null && echo review-ui up

        # db-utilities: build its image once (it runs only when you load a file)
        cd ../db-utilities
        docker compose build
```

The client UI, in its own folder on the VM: `git pull && docker compose up -d --build`.

8. Re-run charts for the new rules

What changed for results: the blank / junk rules (`docs/specials/BLANK_JUNK_LOGIC.md`), the page-type tie (Progress Note first) and rotation. `skip_ocr: true` re-runs every stage on the OCR already stored, so **no Doc Intel bill**. The rotation stage still runs; only a page whose rotation now comes out different is OCR'd again. With `RUN_MODE=queue` the call returns at once and the workers take the charts one by one.

```
        # Ubuntu VM (input_path = the Blob folder the charts came from)
        curl -X POST localhost:8001/api/charts/batch-run -H 'Content-Type: application/json' \
          -d '{"input_type":"blob","container_name":"imaging-pipeline","input_path":"raw_input/Batch1/Run1",
              "output_path":"Processed/Batch1/Run1","skip_ocr":true}'

        :: Windows (Command Prompt), against the VM
        curl -X POST http://172.20.4.170:8001/api/charts/batch-run -H "Content-Type: application/json" -d "
{\"input_type\":\"blob\",\"container_name\":\"imaging-
pipeline\",\"input_path\":\"raw_input/Batch1/Run1\",\"output_path\":\"Processed/Batch1/Run1\",\"skip_ocr\":true}"
```

Add `"chart_list": ["<chart>", ...]` to re-run only some charts. Progress: the call answers with a run id per chart; `curl -s localhost:8001/api/runs/<run id>` (queued / running / done), `curl -s localhost:8001/api/charts/by-name/<chart>`, and `docker compose logs -f worker`.

9. The two utilities (from the Windows laptop)

The laptop reaches the database at 172.20.4.170:5432, so both run there from the core-pipeline venv (`cd core-pipeline && .venv\Scripts\activate.bat`). `DATABASE_URL` set in the window wins over the one in `.env`. Put your password where it says `<password>`.

```
:: Windows (Command Prompt), in the DevOps folder (or the unzipped model-repo-main)
cd /d C:\work\advantmed-devops\core-pipeline
.venv\Scripts\activate.bat
set "DATABASE_URL=postgresql+psycopg://aiuser:<password>@172.20.4.170:5432/imaging_outputs"
set "CLIENT_DATABASE_URL=postgresql+psycopg://aiuser:
<password>@172.20.4.170:5432/imaging_outputs_client"

:: Mark charts "Special" (History and Accuracy filter by tag)
python ..\utilities\tag_chart.py 52743839_44976074 --tag Special
python ..\utilities\tag_chart.py --from-file charts.txt --tag Special
python ..\utilities\tag_chart.py --list-tags

:: The client's ground truth into ours
python ..\utilities\pull_client_ground_truth.py
python ..\utilities\pull_client_ground_truth.py --apply
python ..\utilities\pull_client_ground_truth.py --list-backups
python ..\utilities\pull_client_ground_truth.py --restore gt_backup_<stamp>
```

The first `pull_client_ground_truth.py` is a dry run: rows per table, ours vs the client's. `--apply` backs ours up (schema `gt_backup_<stamp>` + CSVs in `utilities\output\`) and then copies. No venv on the laptop yet: `py -3.12 -m venv .venv`, activate, then `pip install -r requirements-basic.txt`.

The same from the VM:

```
# Ubuntu VM
cd ~/advantmed-imaging-pipeline/core-pipeline && source .venv/bin/activate
export DATABASE_URL='postgresql+psycopg://aiuser:<password>@172.20.4.170:5432/imaging_outputs'
export CLIENT_DATABASE_URL='postgresql+psycopg://aiuser:
<password>@172.20.4.170:5432/imaging_outputs_client'
python ../utilities/tag_chart.py 52743839_44976074 --tag Special
python ../utilities/pull_client_ground_truth.py              # dry run
python ../utilities/pull_client_ground_truth.py --apply
```

Check it in the database:

```
docker exec postgres-db psql -U postgres -d imaging_outputs -c "SELECT count(*) AS pages,
count(DISTINCT chart_name) AS charts FROM page_ground_truth;"
```

Then review-ui → Accuracy View → **Refresh**, and **Download Page Level Accuracy** for the page-by-page Match / No Match file.

10. Later: Cosmos (no code release needed)

When the VM gets a read-write Cosmos key: `DB_BACKEND=both` plus `COSMOS_*` in core-pipeline, review-ui and db-utilities, after `utilities/migrate_postgres_to_cosmos.py --all` has copied the history. A week on `both` with matching counts, then `DB_BACKEND=cosmos`. The API key: `API_KEY_REQUIRED=true` and the same `ORCHESTRATOR_API_KEY` in core-pipeline and review-ui.